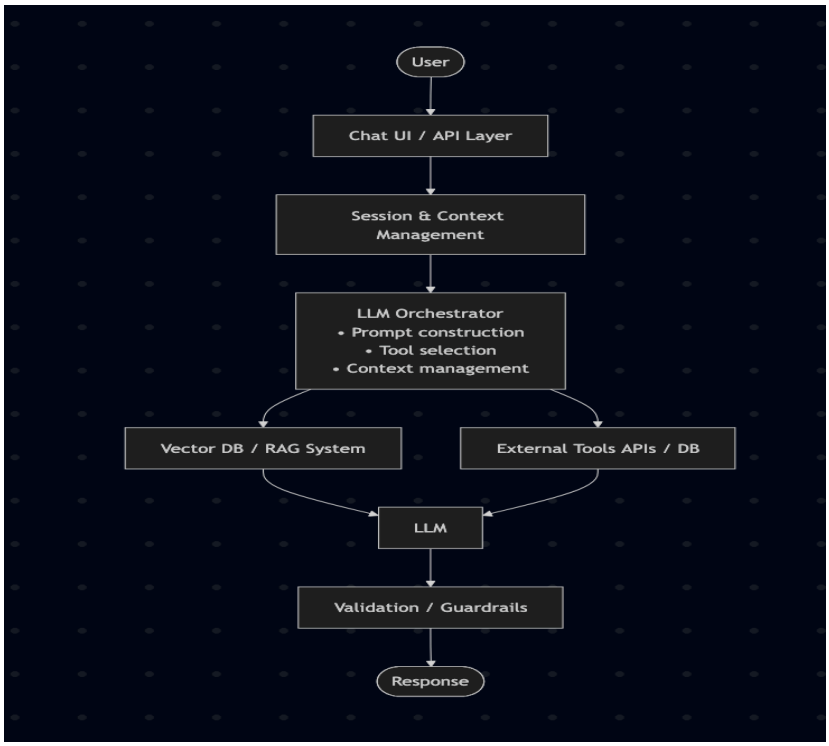
Assignment 2

1. **Where would you rate yourself on (LLM, Deep Learning, AI, ML). A, B, C [A = can code independently; B = can code under supervision; C = have little or no understanding]**

Area	Rating
LLM	B
Deep Learning	A
AI	A
Machine Learning	A

2. **What are the key architectural components to create a chatbot based on LLM? Please explain the approach on a high-level**



Key Architectural Components of an LLM-Based Chatbot

An LLM-based chatbot is more than simply sending a user's message to an LLM and returning the generated response. A practical chatbot usually consists of several components responsible for communication, context management, knowledge retrieval, model inference, tool usage and validation.

A high-level architecture can be represented as:

User -> Chat/API Layer -> Session Management -> LLM Orchestrator -> RAG/Tools -> LLM -> Validation -> Response

User Interface and API Layer

The user interacts with the chatbot through a web application, mobile application or messaging interface. The backend API receives the request, validates it and passes it to the chatbot system. This layer also handles authentication, rate limiting and request-level security.

Session and Conversation Management

The system needs to maintain conversational context. For example, if a user asks:

"What is the refund policy?"

and then asks:

"Does it apply to international orders?"

The chatbot needs the context of the previous question.

Conversation history can be maintained as short-term context, while important long-term information can be stored separately when required.

LLM Orchestrator

The orchestrator is responsible for deciding how a request should be processed.

It can:

- construct the prompt
- provide relevant conversation history
- retrieve external information
- decide whether a tool is required
- call external APIs or databases
- send the final context to the LLM

For a simple chatbot, this logic can be implemented directly in the backend. For a more complex system, an orchestration framework can be used.

Knowledge Retrieval / RAG

A general-purpose LLM may not contain the organization's latest or private information. Retrieval-Augmented Generation (RAG) can be used to provide relevant information to the model.

A typical flow is:

Documents -> Chunking -> Embeddings -> Vector Database -> Similarity Search -> Relevant Context -> LLM

For example, a customer-support chatbot could retrieve the relevant section of a company's return policy before generating its response.

Tools and External Systems

A chatbot may need to perform actions instead of only generating text.

Examples include:

- querying a database
- checking an order
- creating a booking
- calling a payment or CRM API
- searching external information
- performing calculations

The LLM can determine when a tool is required, while the application backend should control what the model is actually allowed to execute.

LLM

The LLM is responsible for understanding the user's request and generating the response using the instructions, conversation context, retrieved information and tool results provided by the application.

The choice of model depends on requirements such as accuracy, latency, context length, cost and privacy.

Validation and Guardrails

The generated output should not always be returned directly to the user.

Depending on the application, the system may validate:

- output format
- required fields
- safety constraints
- permissions
- tool usage
- sensitive information

Structured outputs can also be validated against a schema before being passed to another system.

Monitoring and Evaluation

In production chatbot should also have observability.

Useful metrics include:

- response latency
- token usage
- API failures
- retrieval quality

- tool failures
- user feedback
- hallucination/error rates

This allows the system to be evaluated and improved over time.

Overall Approach

I would start with the simplest architecture required by the problem. For a basic FAQ chatbot, an LLM with conversation history may be sufficient. If the chatbot needs access to private or frequently changing information, I would add RAG. If it needs to perform actions such as booking or querying business systems, I would add controlled tool/API integration.

This keeps the architecture modular and avoids adding components that are not required by the use case.

3. **Please explain vector databases. If you were to select a vector database for a hypothetical problem (you may define the problem) which one will you choose, and why?**

Vector Databases

A vector database is a database designed to store, index and search high-dimensional numerical representations called vectors or embeddings.

An embedding model converts data such as text, images or audio into numerical vectors that capture semantic characteristics of the original data.

For example, two sentences with similar meanings may be represented by vectors that are close to each other in the embedding space:

"How can I reset my password?"

and

"I forgot my password. How do I change it?"

Although these sentences use different words, their embeddings can be semantically similar.

How Vector Search Works

A typical vector-search pipeline consists of the following steps:

1. Documents are collected from the required data sources.
2. Documents are split into smaller chunks.
3. An embedding model converts each chunk into a numerical vector.
4. The vectors and associated metadata are stored in a vector database.
5. When a user submits a query, the query is also converted into an embedding.
6. The vector database searches for vectors that are most similar to the query vector.
7. The most relevant chunks are returned.
8. In a RAG system, these chunks are provided as context to an LLM.
9. The LLM generates the final answer using the retrieved information.

Conceptually:

Documents -> Chunks -> Embeddings -> Vector Database

and during retrieval:

Query -> Embedding -> Similarity Search -> Relevant Chunks -> LLM

Similarity Search

Vector databases use similarity metrics to determine how close two vectors are.

Common approaches include:

- cosine similarity
- Euclidean distance
- dot product

The appropriate metric depends on the embedding model and application.

Vector databases also commonly support metadata associated with each vector. For example:

Vector

- document_id
- department
- document_type
- created_at

- access_level

This allows retrieval to combine semantic similarity with filtering.

Hypothetical Problem: AI Assistant for Oil & Gas Technical Documentation

For the hypothetical problem, I would consider an AI system for the oil and gas industry that helps engineers and operations teams retrieve information from a large collection of technical documents.

An oil and gas organization can have a large amount of unstructured information, including:

- equipment manuals
- maintenance procedures
- operating procedures
- inspection reports
- safety documentation
- engineering documents
- incident reports
- technical specifications
- historical maintenance records

Finding relevant information from these documents using traditional keyword search can become difficult, particularly when engineers describe the same equipment or problem using different terminology.

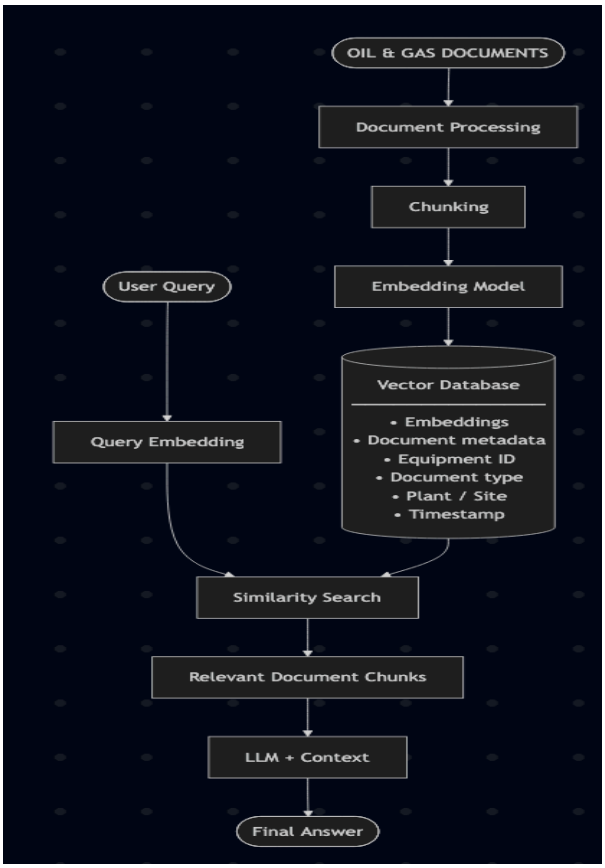
For example, an engineer might ask:

> "What should be checked if the pressure on this compressor is behaving abnormally?"

The relevant information may not contain those exact words. It could instead be described in an equipment manual using terms such as *compressor discharge pressure*, *pressure fluctuation*, *operating limits*, or *troubleshooting procedure*.

This is a situation where semantic search using embeddings can be useful.

Proposed Architecture



Role of the Vector Database

The vector database would store the embeddings generated from document chunks along with metadata.

For example, a stored record could conceptually contain:

- Embedding
- Document ID
- Equipment ID
- Plant / Site
- Document Type
- Section
- Timestamp
- Access Level

This allows the system to perform both semantic retrieval and metadata-based filtering.

For example, an engineer could ask:

"Show me the troubleshooting procedure for compressor C-102 at Plant A."

The retrieval system could combine:

1. semantic similarity to identify relevant troubleshooting information, and
2. metadata filters to restrict the search to `compressor C-102` and `Plant A`.

This is particularly useful because simply finding semantically similar information is not sufficient in an industrial environment. The retrieved information also needs to correspond to the correct equipment, location, document version and potentially the user's access permissions.

Why a Vector Database?

A traditional relational database is well suited for structured information such as:

- Equipment ID
- Equipment Type
- Plant
- Maintenance Date
- Status

However, technical documentation is largely unstructured text.

A vector database allows the system to represent the semantic meaning of document sections as embeddings and retrieve information based on meaning rather than exact keyword matches.

This makes it useful as part of a Retrieval-Augmented Generation (RAG) system.

Vector Database Selection

For this hypothetical system, I would choose Qdrant.

My main reasons would be:

1. Semantic Search

Qdrant can be used to store embeddings and perform similarity search over large collections of document chunks.

This is useful for retrieving technically relevant information even when the user's wording differs from the wording in the source document.

2. Metadata Filtering

The oil and gas use case requires more than semantic similarity.

Documents may need to be filtered by:

- equipment
- plant
- site
- document type
- document version
- date
- access level

For example:

semantic similarity

+

equipment_id = "C-102"

+

plant = "Plant A"

+

document_type = "maintenance_manual"

Combining these constraints can make retrieval much more precise.

3. Separation of Structured and Unstructured Data

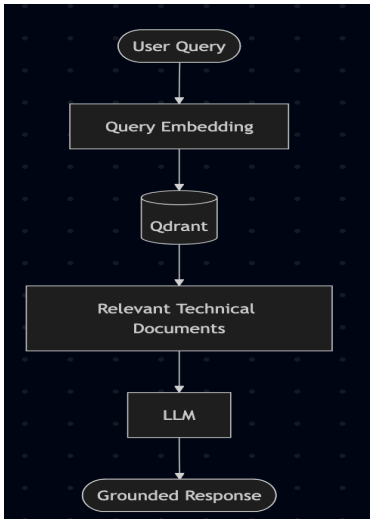
I would not use a vector database as a replacement for the organization's primary relational database.

Structured operational information could remain in a relational database, while Qdrant could handle semantic retrieval from unstructured technical documents.

The application layer could combine information from both systems before passing the relevant context to the LLM.

4. Suitable for RAG

Qdrant can act as the retrieval layer in a RAG architecture:



Important Considerations

For an industrial application, retrieval accuracy and access control are particularly important.

The system should not simply return the nearest vectors and trust the LLM to figure everything out.

I would therefore consider:

- metadata filtering
- document versioning
- access control
- source attribution
- retrieval evaluation
- chunking strategy
- embedding model selection
- hallucination monitoring
- audit logging

The chatbot should also provide references to the source documents or sections used to generate the answer. This allows an engineer to verify the information rather than treating the LLM response as an authoritative source.

Alternative Choices

The choice would depend on the scale and existing infrastructure.

For a small proof of concept, I could use ChromaDB because it is simple to set up and works well for local RAG experimentation.

If the organization already relies heavily on PostgreSQL, pgvector could be attractive because vector search could be integrated into the existing database infrastructure.

For a production-oriented system requiring vector search, metadata filtering and a dedicated retrieval layer, I would choose Qdrant for this hypothetical scenario.

The important point is that the vector database is only one component of the system. The quality of the final AI assistant would also depend heavily on document processing, chunking, embedding quality, retrieval strategy, metadata design, access control and the LLM used for generation.